

Generative AI for Software Engineering: Generative AI Effectiveness and Efficiency in Software Development Documentation

Rohith Gadamsetty
Department of Computer Science
Texas A&M University-Corpus Christi
Corpus Christi, TX, USA
grohith@islander.tamucc.edu

Kadir Alpaslan Demir *
Department of Computer Science
Texas A&M University-Corpus Christi
Corpus Christi, USA
kadiralpaslan.demir@tamucc.edu
kadiralpaslandemir@gmail.com
* Contact Author

Bozhen Liu
Department of Computer Science
Texas A&M University-Corpus Christi
Corpus Christi, USA
bozhen.liu@tamucc.edu

Abstract— This research aims to evaluate the capabilities of Generative Artificial Intelligence (GenAI) in automatically generating software development artifacts, such as project vision, project plan, software requirements specification, software design description, and software test description. The study compares manual artifact generation vs. ChatGPT's automated artifact generation. The research results indicate that ChatGPT can produce accurate, coherent artifacts with significantly reduced effort and time. While the efficiency achieved for software documentation generation utilizing ChatGPT is around 40%, the document quality achieved utilizing ChatGPT was a little lower than what is achieved with the manual approach. ChatGPT demonstrated usefulness across various development tasks, with some limitations, such as the need for human validation in some cases. The findings indicate that GenAI can enhance software documentation efficiency and productivity if integrated appropriately into development workflows. Further research is needed on ChatGPT's real-world applicability and comparison to other AI models. This study provides valuable insights into ChatGPT's potential as an automated software artifact generation tool.

Keywords—Generative AI for Software Engineering, Generative Artificial Intelligence, Generative AI, Software Documentation, Automated Software Documentation, Software Documentation Generation

I. INTRODUCTION

In software development, the creation of accurate and adequate software artifacts is important for fostering collaboration [9],[12], maintaining project transparency [10],[11], and ensuring the delivery of high-quality software products [11],[12]. Software artifacts, such as project vision, plan, requirements, design, and test documents, play a pivotal role in capturing and conveying critical information throughout the development lifecycle [12],[13]. However, despite their significance, the generation of these artifacts is challenging due to time constraints, resource limitations, and the need for domain expertise [14],[15]. The introduction of Large Language Models (LLMs), such as ChatGPT [16], has created opportunities to improve the automation of software development [17]. ChatGPT has shown promising capabilities

in understanding and generating human-like text, sparking interest in its potential applications across multiple domains, including software development.

This research aims to investigate the effectiveness of ChatGPT as an automated tool for software artifact generation. By exploring the strengths and limitations of this LLM, we seek to identify its potential as a development tool in the software development process. This study focuses on evaluating ChatGPT's ability to produce accurate, coherent, and contextually relevant artifacts. To achieve this, we conducted a comparative analysis by creating a sample project and generating the software artifacts manually with human developers. Subsequently, we utilized ChatGPT to generate similar artifacts. The study evaluated the quality of ChatGPT-generated artifacts in comparison to human-crafted ones. Efficiency was measured by analyzing the time required for ChatGPT to generate each artifact. We explored the capability of ChatGPT by applying it to various software development tasks, such as requirement gathering and design documentation. Moreover, we identified potential challenges and limitations associated with ChatGPT's usage. With this research, we aim to contribute to the existing body of knowledge surrounding the effectiveness of AI-powered tools like ChatGPT for automated software development document artifact generation. The insights gained from this research will not only advance our understanding of AI in software engineering but also inform practitioners about the capabilities and limitations of ChatGPT, thereby enabling them to make informed decisions when integrating AI technologies into their software development workflows.

II. RELATED WORKS

The application of large pre-trained language models for automating software engineering activities has gained attention recently [1]. Several studies have explored using LLMs like GPT-3, CodeBERT, and T5 for generating artifacts, including code [18], documentation [19], [20], diagrams [23],[24], and test cases [21],[22]. These models have shown promise for producing higher-quality artifacts with improved efficiency over manual approaches. For code generation specifically,

models like CodeBERT [2] and probabilistic code models [3] have been proposed. CodeBERT is a pre-trained model tailored for programming language processing tasks. The probabilistic code models utilize decision trees for generating code. Other studies have focused on generating UML diagrams from natural language requirements using retrieval and refinement techniques [4]. This allows for transforming textual requirements into visual design diagrams. Additionally, models like GPT-3 have been leveraged to automatically generate code comments and documentation from source code [5], reducing manual documentation efforts. For test case generation, approaches based on T5 [6] and other techniques [7] have been explored to produce test cases from requirements documents. This demonstrates LLM capabilities for generating test artifacts

from high-level specifications. While these works have evaluated models on individual artifact types, a comprehensive assessment across all software engineering artifacts is needed. Our research aims to holistically evaluate ChatGPT across project vision, plan, requirements, design, and testing artifacts.

III. SOFTWARE DEVELOPMENT DOCUMENT ARTIFACTS

The software artifacts that are commonly developed during a software development project are presented in Table I. These documents are developed based on widely accepted software development and documentation standards such as IEEE-15288 [25], IEEE-15289 [26], and MIL-STD-498 [27].

TABLE I. PROJECT DEVELOPMENT DOCUMENTS

Project Artifact / Document	Description
Project Vision Document (PVD)	The Project Vision Document provides stakeholders, including clients and development teams, with a clear and concise overview of the project's objectives, scope, and intended outcomes. PVD outlines the high-level vision and goals of the project, elucidating the problem statement and the proposed solution.
Project Plan Document (PP)	The Project Plan Document serves as a guide for project teams, ensuring a systematic and organized approach to achieving project goals. The document defines project milestones, timelines, resource allocation, a breakdown of tasks and responsibilities, risk management, and communication plans.
Software Requirements Specification (SRS)	The Software Requirements Specification Document lays the foundation for the development process by defining the functional and non-functional (quality) requirements of the software product. The SRS document includes detailed use cases, user stories, and system requirements, providing a clear understanding of the software's features, interfaces, and constraints.
Software Design Description (SDD)	The Software Design Description describes the architectural and technical aspects of the software system. It provides a blueprint for the development team, outlining the structure, components, and interactions of the software.
Software Test Description (STD)	The Software Test Description includes the testing strategies, plans, and procedures for validating the software's functionality and performance. It outlines the test objectives, test cases, and test data, ensuring that each requirement and use case is thoroughly tested and verified.

IV. METHODOLOGY

To compare the effectiveness and efficiency of GenAI utilization in software development artifact generation, we developed software document artifacts for two different projects. The first project is a generic online marketplace project. One of the most well-known application examples is the Amazon online marketplace. The second project is another common application for professional networking social media projects. One of the most popular example applications is LinkedIn. Naturally, such projects require extensive development and documentation. However, for this research, a reduced set of documentation and scope is adequate, which can be the basis for developing such applications.

In the first project, we developed the software development artifacts manually, meaning without the help of any generative AI tool. Furthermore, we developed two versions of the documents to simulate the process of improving and refining processes in real-world projects. We record the time, in other words, the efficiency, for document artifact development processes. We call this process the manual process, which is the traditional way of developing software document artifacts. In this manual process, mainly humans are involved in the process. To enable the comparison and have a benchmark, we assumed that the resulting documents from the manual human-based process have a benchmark quality score of 100. This

score is not a score reflecting the highest quality, but only a score for benchmarking purposes. It is used to compare the manual process with the automated process that utilizes a GenAI tool such as ChatGPT. Our goal is to understand whether the automated process yields better results or not in terms of quality. The document artifacts are developed by a software developer who has 3-5 years of software development experience. The document artifacts are reviewed and refined by an expert software development manager with over 11 years of software development experience. Therefore, the developer and project manager represent common development environment team members.

In the second project, the development of a generic professional networking social media application, we utilized ChatGPT for software document artifact generation. Even though the process may seem like document generation, these processes are crucial for software development. During the development process, application features, requirements, use cases, test cases, and other software artifacts are developed within the documentation process. Therefore, except for code, many crucial artifacts are developed with these processes. Like the first project, we developed two versions of documentation. The second version has gone through improvement and refinement processes. We also utilized refined prompts in the second version based on our experiences from the first version, as any software development process would benefit from the

refined use of automated tools. During the development process, we record the time necessary to develop the artifacts. At the end of the process, we assess the documents for quality and compare the quality based on the benchmark from the manual process.

A. Manual Process for a Generic Online Marketplace Project

In the manual process for a generic online marketplace project documentation, we developed two versions of the documents: Version 1 (v1) and Version 2 (v2). These versions were designed to showcase incremental improvements. In the v1 documents, we laid the foundation for the project, providing a comprehensive overview of the project's objectives, scope, and key features in the Project Vision Document. The Project Plan Document outlined the initial timeline, milestones, and resource allocation for the project's development. The Project Software Requirements Specification (SRS) captured the functional and non-functional requirements, along with use cases, forming the basis for the application's development.

As the development process progressed, we enhanced and refined the documents. For the v2 documents, we iteratively improved the content by adding more features, requirements, use cases, design decisions, and test cases. Project Vision Document v2 reflected these enhancements, outlining the expanded functionalities and improvements made over the initial version. In the Project Software Requirements Specification (SRS) v2, we captured the newly identified requirements and extended use cases to accommodate enhanced features. Additionally, we addressed design constraints and provided a more comprehensive understanding of the system's behavior. The Project Software Design Description (SDD) v2 included an improved architectural design, incorporating design refinements to accommodate the new features and improve the user experience. The SDD outlined the interactions among different components and provided an updated user interface design. For comprehensive validation, we developed a more extensive set of test cases in the Project Software Test Document v2. These test cases covered the additional features and functionalities, ensuring thorough testing of the updated application. The manual process for both v1 and v2 versions of the generic online marketplace website project documentation ensured a well-documented, iterative development lifecycle that accommodated evolving requirements and delivered a robust and user-centric application.

B. Automated Process for a Generic Professional Networking Social Media Project

In the automated process for the Generic Professional Networking Social Media project documentation, we utilized ChatGPT to generate two versions of documents: version 1 (v1) and version 2 (v2). The automated process allowed for improvements by incorporating additional features, use cases, design decisions, and test cases iteratively. Starting with the input of key requirements and specifications for both versions of the professional networking application, ChatGPT automatically generated the initial Project Vision Document. This document provided a concise and contextually relevant overview of the project's objectives and scope.

As the development process progressed, we fed the refined vision, features, and requirements back into ChatGPT for the

second version of the documents. ChatGPT incorporated these improvements and generated an updated Project Vision Document, including enhanced features and functionalities. The automated generation process facilitated rapid iteration, allowing us to iterate over the content and continuously improve the documentation. The Project Software Requirements Specification (SRS) v1 was initially generated, outlining the functional and non-functional requirements, along with key use cases. As new features and requirements identified, we integrated this information into the input data for ChatGPT to produce the SRS v2. ChatGPT captured the new requirements and use cases, reflecting the iterative improvements on the professional social media networking application. The Project Software Design Document (SDD) v1 was automatically generated to provide the initial architectural design and user interface details. As the application development evolved with additional features, we used ChatGPT to create an updated SDD, including the revised architectural design and accommodating the new enhancements. ChatGPT demonstrated its adaptability in processing the input data and generating coherent design documentation. Finally, the Project Software Test Document v1 was created, detailing the test cases for the initial version. For the v2 version, we integrated feedback and new test scenarios into the input for ChatGPT to generate a more comprehensive and updated set of test cases. The automated process enabled rapid test case development iterations. Overall, the automated ChatGPT process streamlined the professional networking social media application project documentation, facilitating efficient iteration and continuous improvement throughout the development lifecycle.

V. RESEARCH RESULTS

We encountered several noteworthy experiences that shed light on the potential of AI-driven technologies in software development. First, we observed a substantial reduction on average, such as 42%, in the time required for generating software artifacts, as presented in Fig. 1. The automated process demonstrated significant efficiency in producing documents like the Project Vision Document, Project Plan Document, Software Requirements Specification, Software Design Description, and Software Test Description. Compared to the manual process, which involved extensive efforts from developers, ChatGPT significantly accelerated the generation of these artifacts, saving valuable time for developers and project stakeholders.

The effectiveness of ChatGPT in producing coherent and relevant software artifacts was evident, as presented in Fig. 2. ChatGPT demonstrated its capacity to understand and synthesize complex information, resulting in documents closely aligned with human-crafted ones. In terms of accuracy, ChatGPT successfully captured essential features, design decisions, use cases, and test cases. While there were instances where human expertise was still needed to validate and fine-tune the artifacts, overall, ChatGPT's effectiveness in generating documentation was promising to streamline the software development process.

The development of the Project Vision Document for the Generic Professional Networking Social Media project was

supported by ChatGPT. First, the goal of the application and the main features needed were provided as input to ChatGPT. Then, ChatGPT enhanced and improved the project vision based on this initial input. Using iterative prompting techniques, the vision document versions were improved until they were satisfactory.

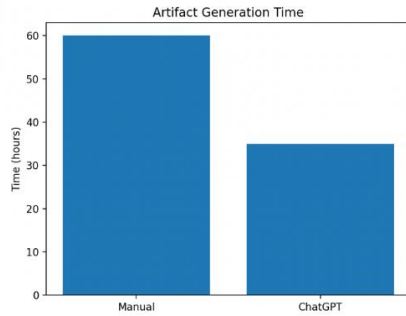


Fig. 1. Software Development Document Artifact Generation Time for Manual and Automated (ChatGPT) Processes

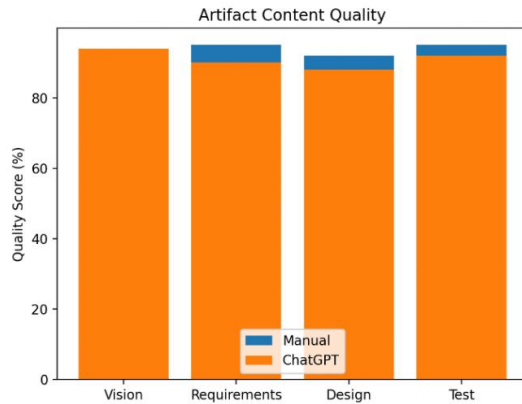


Fig. 2. Software Development Document Artifact Quality for Manual and Automated (ChatGPT) Processes

In the Project Plan (PP) development, estimating required resources and identifying risks were challenging tasks. While ChatGPT assisted with resource allocation, budget estimation, identifying potential risks, and suggesting risk mitigation strategies, ChatGPT was not able to produce a satisfactory project plan without human guidance. The reason was that developers have a significant portion of inherent project planning information that is not apparent to ChatGPT. For example, when managers plan for a project, they inherently consider the skills and capabilities of the development teams. Without the inputs of such inherent considerations, ChatGPT was unable to produce satisfactory project plans. Therefore, what was produced by ChatGPT was quite generic, which needs to be customized for the project. Note that Fig. 2 does not include the project plan document, as we did not find the resulting document satisfactory.

For the Software Requirements Specification (SRS), gathering comprehensive requirements and prioritizing tasks were time-consuming tasks. ChatGPT helped by providing

specific requirements for individual modules, suggesting features based on competitor analysis, and offering priorities based on previous ideas and implementation methods. Furthermore, for complex scenarios like order tracking, ChatGPT assisted in documenting requirements, providing insights, and addressing system behaviors and interactions.

Developing the Software Design Description (SDD) was challenging due to identifying a suitable architecture and handling error scenarios effectively. ChatGPT assisted in exploring architectural design options and suggested best practices for scalability and maintainability. It also provided strategies for handling errors, ensuring robustness in the system design.

In developing the Software Test Description (STD), challenges included writing comprehensive test case steps and ensuring test coverage. ChatGPT provided clearer information on writing test cases with success and failure scenarios, helping developers and business teams evaluate the application better. Additionally, it offered more test cases and divided complex test cases into smaller modules to improve test coverage.

VI. CONCLUSION

This research focuses on assessing the effectiveness of ChatGPT as a tool for software development artifact generation, and the findings provide valuable insights into the potential applications of AI-driven technologies in this domain. ChatGPT can efficiently generate software artifacts, including the Project Vision Document, Project Software Requirements Specification, Project Software Design Document, and Project Software Test Document. By comparing the outputs of ChatGPT with manually crafted artifacts, we observed that the LLM consistently delivered coherent and contextually relevant documentation. This suggests that ChatGPT can significantly reduce the time and effort required for generating software artifacts, streamlining the software development process, and improving overall productivity. By leveraging AI-powered tools like ChatGPT, development teams can efficiently generate comprehensive and well-structured documentation, leading to improved collaboration, reduced development time, and enhanced software quality. However, practitioners must strike a balance between the benefits of automation and the need for human expertise in validating and refining artifacts. As AI technologies continue to evolve, further research can lead to more refined models that effectively complement human efforts in creating software artifacts, ultimately improving the software development process for greater efficiency and innovation.

VII. FUTURE WORK

Future work in this research area can focus on refining ChatGPT's capabilities, enhancing collaboration between AI and human experts, addressing limitations, and exploring its practical applicability in real-world software development projects. These efforts can further advance the integration of AI-driven technologies in the software development domain, leading to more efficient and innovative development practices.

VIII. ACKNOWLEDGEMENTS AND DISCLAIMERS

The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily

representing the official policies or endorsements, either expressed or implied, of any affiliated organization or government.

IX. REFERENCES

- [1] T. B. Brown et al., "Language models are few-shot learners," arXiv preprint arXiv:2005.14165, 2020.
- [2] Z. Xu et al., "Codebert: A pretrained model for programming and natural languages," arXiv preprint arXiv:2109.04338, 2021.
- [3] V. Raychev, M. Vechev and E. Yahav, "Code completion with statistical language models," in Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation, 2014, pp. 419–428.
- [4] M. Di Federico et al., "Generating UML diagrams from natural language requirements," in Proceedings of the 2021 33rd ACM/IEEE International Conference on Automated Software Engineering, 2021, pp. 836–847.
- [5] Z. Chen et al., "Evaluating large language models for code comment generation," arXiv preprint arXiv:2103.11835, 2021.
- [6] X. Lin, et al., "GPT-TE: An intelligent automatic test case generation tool based on T5," Applied Sciences, vol. 12, no. 11, p. 5754, 2022.
- [7] Y. Han and H. Yin, "A survey of natural language generation in software engineering," in Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2021, pp. 1073–1084.
- [8] M. Yazdani and D. Draheim, "A review of generative language models in software engineering," in IEEE/ACM 43rd Int. Conf. on Software Engineering: Software Engineering in Society (ICSE-SEIS), 2021, pp. 155–158.
- [9] Constantino, Kattiana, Shurui Zhou, Mauricio Souza, Eduardo Figueiredo, and Christian Kästner. "Understanding collaborative software development: An interview study." In Proc. of the 15th Int. Conf. on Global Software Engineering, pp. 55–65. 2020.
- [10] Christopher S. Timperley, Lauren Herckis, Claire Le Goues, Michael Hilton, "Understanding and Improving Artifact Sharing in Software Engineering Research", arXiv preprint arXiv:2008.01046.
- [11] Hossain, E., Babar, M.A., Paik, H.Y. and Verner, J., 2009, December. Risk identification and mitigation processes for using scrum in global software development: A conceptual framework. In 2009 16th Asia-Pacific Software Engineering Conference (pp. 457–464). IEEE.
- [12] Ko, Amy J., Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. "An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks." IEEE Transactions on software engineering 32, no. 12 (2006): 971–987.
- [13] Hackos, JoAnn T. Information development: Managing your documentation projects, portfolio, and people. John Wiley & Sons, 2007.
- [14] Nazar, Najam, Yan Hu, and He Jiang. "Summarizing software artifacts: A literature review." Journal of Computer Science and Technology 31, no. 5 (2016): 883–909.
- [15] Ghazi, Parisa, and Martin Glinz. "Challenges of working with artifacts in requirements engineering and software engineering." Requirements engineering 22, no. 3 (2017): 359–385.
- [16] ChatGPT, <https://chatgpt.com/>, Accessed on Sept. 1, 2025.
- [17] Fan, Angela, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. "Large language models for software engineering: Survey and open problems." In 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), pp. 31–53. IEEE, 2023.
- [18] Poesia, Gabriel, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. "SynchroMesh: Reliable code generation from pre-trained language models." arXiv preprint arXiv:2201.11227 (2022).
- [19] Khan, Junaed Younus, and Gias Uddin. "Automatic code documentation generation using gpt-3." In Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, pp. 1–6. 2022.
- [20] Feng, Zhangyin, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou et al. "Codebert: A pre-trained model for programming and natural languages." arXiv preprint arXiv:2002.08155 (2020).
- [21] Mathur, Alok, Shreyaan Pradhan, Prasoon Soni, Dhruvil Patel, and Rajeshkannan Regunathan. "Automated test case generation using t5 and GPT-3." In 2023 9th International Conference on Advanced Computing and Communication Systems (ICACCS), vol. 1, pp. 1986–1992. IEEE, 2023.
- [22] Siddiq, Mohammed Latif, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinícius Carvalho Lopes. "Using large language models to generate junit tests: An empirical study." In Proceedings of the 28th international conference on evaluation and assessment in software engineering, pp. 313–322. 2024.
- [23] De Bari, Daniele. "Evaluating large language models in software design: A comparative analysis of uml class diagram generation." PhD diss., Politecnico di Torino, 2024.
- [24] Ferrari, Alessio, Sallam Abualhajjal, and Chetan Arora. "Model generation with LLMs: From requirements to UML sequence diagrams." In 2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW), pp. 291–300. IEEE, 2024.
- [25] IEEE/ISO/IEC 15288-2023 ISO/IEC/IEEE International Standard – Systems and software engineering–System life cycle processes.
- [26] IEEE/ISO/IEC 15289-2019 ISO/IEC/IEEE International Standard – Systems and software engineering - Content of life-cycle information items (documentation).
- [27] DeWeese, Perry R., and Virginia Center of Excellence for Software Reuse and Technology Transfer Herndon. "MIL-STD-498 Military Standard: Software Development and Documentation. Version 01.00. 00." (1994): 0037